

Chai aur Code - React Videos 1 to 4

Important concepts, revision notes, commands and code examples

How to use these notes: Read the concept, type the code yourself, then explain the flow without looking at the page. Do not try to memorize every line.

Video 1 - React JS Roadmap

Goal: Understand what React is, why it is used, and how the course will be learned.

What is React?

React is a JavaScript library used to build user interfaces (UI). Its core idea is building UI from reusable components.

Why React?

Component-based UI, reuse, efficient UI updates, and a large ecosystem.

React is a library

React mainly focuses on the UI layer. Other tools can be added for routing, state management, APIs, etc.

Learning approach

Use a practical loop: Watch -> Understand -> Code -> Experiment -> Build.

Key note: Strong JavaScript fundamentals make React much easier.

Video 2 - Create React Projects

Goal: Learn how a React project is created and the main commands used with Vite.

1. Create a project with Vite

```
npm create vite@latest  
# choose: React -> JavaScript  
# then choose your linter (for the tutorial setup)
```

2. Install dependencies

```
npm install
```

3. Start the development server

```
npm run dev
```

Important files and folders

Item	What it is for
package.json	Project metadata, dependencies and npm scripts.
node_modules	Installed project dependencies. Do not edit manually.
src/	Main source-code folder where most React work happens.
public/	Static/public assets.
index.html	HTML entry page loaded by the browser.

Remember: npm manages packages; npx runs packages/commands. For your current setup, Vite is the project-development tool being used.

Video 3 - Understand React Flow and Structure

Main idea: Understand how a React app starts in the browser and reaches your components.

The core flow

```
index.html
|
v
main.jsx
|
v
<App />
|
v
App.jsx / components
|
v
JSX -> rendered UI
|
v
Browser DOM
```

index.html

In a Vite React app, index.html contains a root element such as `<div id="root"></div>` and loads main.jsx.

main.jsx

```
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
```

```
createRoot(document.getElementById('root')).render(
  <App />
)
```

`document.getElementById('root')` finds the HTML root element. `createRoot(...)` connects React to that container. `render(<App />)` renders the App component there.

App.jsx

```
function App() {
  return <h1>Hello React</h1>
}
```

```
export default App
```

A component can return JSX. `export default` allows another file to import that component.

Component naming

```
<div />    // HTML element
<h1 />    // HTML element
<App />    // React component
<Navbar /> // React component
```

Important: Lowercase JSX names are treated as intrinsic/HTML elements. Custom React components conventionally start with a capital letter.

Connection to your earlier error

If code tries `container.appendChild(...)` while container is null, JavaScript throws an error such as: `Cannot read properties of null (reading 'appendChild')`. This usually means the expected DOM element was not found.

Video 3 must-remember: index.html -> main.jsx -> createRoot -> App component -> UI.

Video 4 - Create Your Own React Library and JSX

Main idea: Understand, at a simplified teaching level, how JSX can become a JavaScript representation and how a renderer can create DOM nodes.

1. JSX is not HTML

JSX looks like HTML, but it is JavaScript syntax used to describe UI. It is transformed into JavaScript by the tooling.

```
<h1>Hello React</h1>
```

```
// conceptually similar to:  
React.createElement('h1', null, 'Hello React')
```

2. createElement concept

```
createElement(type, props, children)
```

```
// example:  
createElement(  
  'a',  
  { href: 'https://example.com' },  
  'Click Me'  
)
```

type = element/component type; props = properties/attributes; children = content or nested elements.

3. DOM creation and rendering

```
const dom = document.createElement(type)  
// set props  
// add children  
container.appendChild(dom)
```

This is a teaching model, not the complete implementation of real React.

4. JSX can contain JavaScript

```
const name = 'Arpan'  
  
<h1>Hello {name}</h1>
```

Curly braces inside JSX let you insert JavaScript expressions.

5. Full mental model

```
JSX  
|  
v  
JavaScript representation  
|  
v  
Renderer  
|  
v  
DOM nodes  
|  
v  
Browser UI
```

Do not memorize the custom React library line-by-line. Understand the flow and the purpose of createElement, props, children, DOM creation and rendering.

Rapid Revision - Videos 1 to 4

Video	Remember these
1	React = JavaScript library for UI; reusable components; project-based learning.

Video	Remember these
2	Vite setup: npm create vite@latest -> npm install -> npm run dev; know package.json and src/.
3	Browser flow: index.html -> main.jsx -> createRoot -> -> components -> UI.
4	JSX is JavaScript syntax; conceptually JSX -> createElement representation -> renderer -> DOM.

Practice rule: After Video 6, start building your own small React projects, but practice each new concept as you learn it.